RAG-BASED CUSTOMER SUPPORT ASSISTANT

Low-Level Design (LLD)

E-Commerce Support Intelligence System

Internship Project | Version 1.0

1. Module-Level Design

The system is divided into eight distinct modules. Each module has a single responsibility and communicates with adjacent modules through well-defined interfaces.

Module	Responsibility & Implementation
Document Processing Module	File: ingest.py. Accepts a PDF path, uses PyPDFLoader to extract text page-by-page, and passes the resulting Document list to the Chunking Module. Runs once offline.
Chunking Module	File: ingest.py. Receives List[Document] from the processing module. Uses RecursiveCharacterTextSplitter with chunk_size=500 and chunk_overlap=100 to produce smaller Document objects. The recursive splitter respects natural boundaries (paragraphs, sentences) before splitting mid-sentence.
Embedding Module	File: ingest.py. Wraps SentenceTransformerEmbeddings(model='all-MiniLM-L6-v2'). Converts each chunk's page_content string into a float[384] vector. Runs locally — no API key. Called by ChromaDB's from_documents() method automatically.
Vector Storage Module	File: ingest.py + retriever.py. ChromaDB stores embeddings + chunk text + metadata. Persists to chroma_db/ directory. Exposes a retriever interface via as_retriever(search_kwargs={'k': 4}).
Retrieval Module	File: retriever.py. Loads the persisted ChromaDB instance and returns a configured retriever object. Called once at graph build time. The retriever is injected into the LangGraph nodes that need it.
Query Processing Module	File: graph.py. The intent_node uses the LLM to classify the query. The generate_node passes the query and retrieved chunks to the LLM with a strict system prompt. The relevance_check_node validates chunk usefulness before generation.
Graph Execution Module	File: graph.py. LangGraph's StateGraph wires all nodes and conditional edges. Compiled once at module import. run_rag() is the public API — it accepts a query string and returns the completed GraphState dict.
HITL Module	File: graph.py. Two nodes trigger human escalation: critical_escalation_node (pre-retrieval, for flagged queries) and hitl_node (post-generation, for low-confidence answers). Both set needs_human=True and route='escalate' in state. In production, these nodes would dispatch tickets to a support CRM.

2. Data Structures

2.1 Document Representation (LangChain Document)

LangChain Document Object (per page, post-load)

```
class Document:
    page_content: str      # Full text of one PDF page
    metadata: dict         # { 'source': 'knowledge_base/faq.pdf', 'page': 3 }
```

2.2 Chunk Format (post-splitting)

Chunk Object (LangChain Document, post-split)

```
class Document:
    page_content: str      # 500-char text fragment
    metadata: dict         # { 'source': str, 'page': int }

# Example:
# page_content: 'Returns must be initiated within 30 days of delivery.'
# metadata: { 'source': 'knowledge_base/faq.pdf', 'page': 4 }
```

2.3 Embedding Structure (ChromaDB internal)

ChromaDB stored record per chunk

```
{
  'id':         str,           # Auto-generated UUID
  'embedding':  List[float],   # 384-dimensional vector (all-MiniLM-L6-v2)
  'document':   str,           # Raw chunk text (page_content)
  'metadata':   {              # Passed-through from Document.metadata
                  'source': str,
                  'page':   int
                }
}
```

2.4 Query-Response Schema (app.py session message)

Message dict stored in st.session_state.messages

```
# User message
{ 'role': 'user', 'content': str }


# Assistant message
{
  'role':    'assistant',
  'content': str,           # Final answer text shown to user
  'route':   str,           # 'answer' | 'escalate' | 'clarify' | 'out_of_scope'
  'intent':  str,           # 'in_scope' | 'critical' | 'ambiguous' |
'out_of_scope'
}
```

2.5 Graph State Object (LangGraph TypedDict)

GraphState — passed between every node in the graph

```
class GraphState(TypedDict):
    query:            str            # Original user input
    intent:           str            # Set by intent_node
    intent_reason:    str            # LLM's one-sentence explanation
    retrieved_chunks: List[str]      # Set by retrieve_node
    chunks_relevant:  bool           # Set by relevance_check_node
    answer:           str            # Set by generate/hitl/clarify/scope nodes
    confidence:       str            # 'high' | 'low' — set by generate_node
    route:            str            # Final route label for UI display
    needs_human:      bool           # Triggers HITL when True
    human_response:   Optional[str]  # Reserved for future human agent replies
```

3. Workflow Design (LangGraph)

3.1 Node Definitions

Node Name	Type & Behaviour
intent_node	LLM Node. Sends query to Groq at temperature=0 with a JSON-format system prompt. Parses response into { intent, reason }. Falls back to 'in_scope' if JSON parse fails. This is the single entry point for all queries.
out_of_scope_node	Rule Node. Returns a fixed polite rejection message. Sets route='out_of_scope', needs_human=False. No LLM call needed.
clarify_node	LLM Node. Sends the vague query to the LLM with a prompt instructing it to ask ONE clarifying question. Returns a natural-language clarification request.
critical_escalation_node	Rule Node. Returns a high-priority escalation message with a 1-hour SLA. Sets route='escalate', needs_human=True. Bypasses all RAG steps.
retrieve_node	Retrieval Node. Calls retriever.invoke(query) to get the top-4 ChromaDB chunks. Stores them as List[str] in state['retrieved_chunks'].
relevance_check_node	LLM Node. Sends top-2 chunks + query to LLM, asks for 'relevant' or 'irrelevant'. Sets state['chunks_relevant'] based on response. Falls back to True on error.
generate_node	LLM Node. Concatenates all chunks as context. Sends context + query to LLM with e-commerce support system prompt. Checks response for low-confidence signals. Sets answer, confidence, needs_human.
hitl_node	Rule Node. Sets route='escalate'. Replaces LLM answer with escalation message. Triggered when generate_node sets needs_human=True or relevance_check returns False.
output_node	Pass-through Node. Sets route='answer'. Returns state unchanged. Terminal node for successful answers.

3.2 Edge Definitions

LangGraph edge configuration

```
# Entry
workflow.set_entry_point('intent')



# Conditional: after intent
```

```
workflow.add_conditional_edges('intent', route_after_intent, {
    'out_of_scope':        'out_of_scope',
    'clarify':             'clarify',
    'critical_escalation': 'critical_escalation',
    'retrieve':            'retrieve',
})

# Linear: retrieve -> relevance_check
workflow.add_edge('retrieve', 'relevance_check')

# Conditional: after relevance check
workflow.add_conditional_edges('relevance_check', route_after_relevance, {
    'generate': 'generate',
    'hitl':     'hitl',
})

# Conditional: after generation
workflow.add_conditional_edges('generate', route_after_generate, {
    'hitl':   'hitl',
    'output': 'output',
})

# Terminal: all leaf nodes -> END
for node in ['out_of_scope','clarify','critical_escalation','hitl','output']:
    workflow.add_edge(node, END)
```

3.3 State Transitions

State Field	Set By Node → Value
intent	intent_node → 'in_scope' | 'out_of_scope' | 'ambiguous' | 'critical'
intent_reason	intent_node → one-sentence explanation string
retrieved_chunks	retrieve_node → List of 4 text strings from ChromaDB

chunks_relevant	relevance_check_node → True | False
answer	generate_node / hitl_node / clarify_node / out_of_scope_node / critical_escalation_node → final answer string
confidence	generate_node → 'high' | 'low'
needs_human	generate_node / critical_escalation_node → True | False
route	output_node / hitl_node / clarify_node / out_of_scope_node / critical_escalation_node → route label string

4. Conditional Routing Logic

4.1 Intent Classification (route_after_intent)

The LLM classifier uses this system prompt to choose between the four intents:

- out_of_scope: Questions unrelated to e-commerce — weather, general knowledge, math, cooking
- ambiguous: Message is too vague to act on — missing key details, typically 1-4 words with no question word
- critical: Requires immediate human intervention — billing disputes, fraud, account compromise, refund requests, legal threats
- in_scope: Normal e-commerce support questions — orders, shipping, returns, accounts, products

The classifier returns a JSON object: { "intent": "...", "reason": "..." }. Temperature is set to 0 for maximum determinism. Invalid JSON or unknown intent values default to 'in_scope' to avoid silent failures.

4.2 Relevance Check Routing (route_after_relevance)

After retrieval, the system verifies that the retrieved chunks are actually relevant to the query before generating an answer. This prevents hallucination when ChromaDB returns chunks that score above the similarity threshold but do not contain the answer. The LLM is asked to respond with 'relevant' or 'irrelevant' only.

- If relevant → proceed to generate_node
- If irrelevant → skip generation, go directly to hitl_node

4.3 Confidence Check Routing (route_after_generate)

After generation, the system scans the LLM's answer for low-confidence signals. If any are found, needs_human is set to True and the query is escalated.

Low-confidence signal phrases (checked case-insensitively)
`'INSUFFICIENT_CONTEXT'  - LLM's explicit no-answer signal`
`'i don't have'  - LLM expressing ignorance`
`'i do not have'`
`'not in the context'`

```
    'i cannot find'
    'no information'
    'i'm unable'
    'i am unable'
    'cannot answer'
    'don't know'
    'not sure'
```

5. HITL Design

5.1 When Escalation is Triggered

Trigger	Condition
Critical Intent	intent_node classifies query as 'critical'. Happens before retrieval. No RAG pipeline runs. Example: 'I was charged twice', 'my account was hacked', 'I want a refund'.
Irrelevant Chunks	relevance_check_node returns chunks_relevant=False. Happens after retrieval. Means ChromaDB found chunks but they do not answer the query. Escalates to avoid generating a hallucinated answer.
Low Confidence Answer	generate_node detects a low-confidence signal in the LLM's response. Happens after generation. The LLM itself signals it cannot answer from the given context.

5.2 What Happens After Escalation

- The escalation node sets route='escalate' and needs_human=True in GraphState
- A user-facing message is returned explaining that a human agent will respond
- For critical escalations: the message includes a 1-hour SLA commitment
- For low-confidence escalations: the message explains the AI couldn't find a reliable answer
- The Streamlit UI renders the message in an amber/red bubble with an 'Escalated' status pill
- The sidebar session counter increments the escalation count

5.3 How Human Response is Integrated (Current vs Future)

Version	Behaviour
Current (v1.0)	human_response field exists in GraphState but is not populated. The system simulates escalation by displaying a message to the user. No real ticket is dispatched.
Future (v2.0)	Integrate a CRM webhook (Zendesk, Freshdesk) in the escalation node. When needs_human=True, POST the query + conversation history to the CRM API, receive a ticket ID, and store it in the session. Poll for human response and inject it back into the chat when available.

6. API / Interface Design

6.1 Public API — graph.py

run_rag(query: str) -> dict

```
Input:  query: str   # Raw user text


Output: GraphState dict containing:
  {
    'query':            str,        # Original input
    'intent':           str,        # Classified intent
    'answer':           str,        # Text to display to user
    'route':            str,        # 'answer'|'escalate'|'clarify'|'out_of_scope'
    'confidence':       str,        # 'high' | 'low'
    'needs_human':      bool,       # True if escalated
    'retrieved_chunks': List[str],  # Chunks used (empty if bypassed)
    'chunks_relevant':  bool,
    'intent_reason':    str,
    'human_response':   None,       # Reserved
  }
```

6.2 Streamlit UI Interaction Flow

UI Event	Code Action
User types query + presses Enter	st.chat_input() captures text → appended to st.session_state.messages
Thinking animation displayed	st.empty() placeholder shows animated dots while run_rag() executes
run_rag() completes	Result dict unpacked. answer and route extracted.
Message stored	{ role, content, route, intent } appended to session messages
st.rerun() called	Page re-renders. All messages replayed from session state.
User clicks suggestion chip	st.button() sets st.session_state.chip_query. On next render, chip_query is processed exactly like typed input.
User clicks Clear	st.session_state.messages reset to []. Counts reset to zero.

7. Error Handling

Error Scenario	Handling Strategy
chroma_db/ directory not found	app.py checks os.path.exists('chroma_db') at startup. Displays a red error box with instructions to run ingest.py. Calls st.stop() to halt the app.
PDF not found during ingestion	PyPDFLoader raises FileNotFoundError. The ingest.py script will crash with a clear error message. User must verify the PDF_PATH constant and file location.
No chunks retrieved from ChromaDB	retrieve_node returns empty list. relevance_check_node detects empty chunks and sets chunks_relevant=False. Query is routed to hitl_node.
Chunks retrieved but irrelevant	relevance_check_node LLM call returns 'irrelevant'. chunks_relevant set to False. Routes to hitl_node, preventing hallucination.
LLM intent classifier returns invalid JSON	classify_intent() wraps the call in try/except. On any exception or unrecognised intent value, defaults to 'in_scope'. This ensures the RAG pipeline still runs rather than crashing.
LLM generate_node raises exception	generate_node catches exceptions in try/except. On failure, answer is set to 'INSUFFICIENT_CONTEXT', which triggers low-confidence routing to hitl_node.
Groq API rate limit exceeded	Groq returns a 429 error. The LangChain Groq wrapper raises an exception. Currently unhandled — the Streamlit app will show an error. Future fix: add exponential backoff retry with tenacity library.
relevance_check LLM call fails	try/except in relevance_check_node catches failure. chunks_relevant defaults to True to avoid unnecessary escalation.